



COMPUTER SCIENCE 180A ALGORITHMS

PROGRAMMING ASSIGNMENT 3 – EXACT COVER SOLVER

Develop an Exact Cover Solver for solving Sudoku. The solver should be capable of taking any instance of an exact cover problem and returning as many possible covers as desired.

1. Translating Exact Cover into U and F

Given a properly formatted exact cover problem where:

- **First line:** Contains the names of the items
 - Each item name will be at most 8 ascii characters
 - Each item name is unique
 - There is always at least one item
 - The item names will be separated by a single space
- **Subsequent lines:** Represent the options, where each option is a subset of the items
 - Every option is composed of item name(s) from the first line (at least one)
 - There is no duplicate item name within an option
 - The item names will be separated by a single space
 - Every option is distinct from one another

The goal is to convert this text file into two parallel arrays:

- **Item[] U:** Contains the horizontal doubly linked list of items, with an additional root item at index 0
- **Node[] F:** Contains nodes representing the items and the options, separated by spacer nodes

1.a. Data Structures

To translate the input into U and F, the following data structures are utilized:

Item

Each Item contains:

- **String name:** the item's name (taken from the first line of the input file)
- **Int left:** index of the previous item in the circular doubly linked list of items
- **Int right:** index of the next item in the list

These fields form the **horizontal circular doubly linked list** of items used by Algorithm X to choose which item to cover next.

Node

Each option is represented as a list of nodes. The nodes are in vertical circular doubly linked lists with respect to each item. Additionally the vertical lists each include an item header node.

Each Node contains:

- **Int top**: interpreted differently based on the node type
- **Int up**: index of the node above it in its vertical list, with exception of SpacerNodes
- **Int down**: index of the node below it its vertical list, with exception of SpacerNodes

1.b. Types of Nodes

The Node[] array F contains **three distinct types of nodes**:

ItemHeaderNodes

These Nodes appear in F at the same **indices** as their corresponding Item objects in U.

They represent the headers of the vertical lists for each item

- **Int top**: stores the number of OptionNodes currently linked in that item's vertical list (excluding the header node itself). This value can change when other items are covered/uncovered

OptionNodes

These represent occurrences of an item within an option

- **Int top**: stores the index of the **ItemHeaderNode** corresponding to the item this node belongs to (In particular, top is always greater than zero and never changes)

Note: nodes in the same option appear **sequentially in F**, and a SpacerNode marks the end/beginning of each option.

SpacerNodes

These represent the beginning/end of an option in F and allows for the traversal of all nodes in an option without horizontal pointers

- **Int top**: set to a value that is distinguishable from OptionNodes (e.g. 0 or -1 since OptionNodes always have top > 0)
- **Int up**: index of the first OptionNode in the option before this spacerNode
- **Int down**: index of the last OptionNode in the option after this spacerNode

Note: For the last SpacerNode, down can be set to any value as it won't be used. Similarly for the first SpacerNode and its up value.

1.c. Translating

Reading and translating the input to U and F will generally involve two distinct steps, initializing U with items, and then updating F to have each option.

Initializing U

This step requires having a root item at index 0 of U and for every item name in the first line of input, appending an item to the end of the doubly linked list. Additionally, F will be filled in with ItemHeaderNodes representing an item with zero options associated with it.

Updating F

As options are read from the input, F should be updated to have OptionNodes that are added to the end of the vertical doubly linked lists of the respective ItemHeaderNodes as well as ensuring each option ends (and begins) with a SpacerNode. To find the index of the ItemHeaderNode, it is sufficient to do a linear scan of U to find a matching itemName as the bottleneck of the performance will be on solving the exact cover which is exponential.

Example

Given the following input:

```
a b c d e f g
c e
a d g
b c f
a d f
b g
d e g
```

After processing the first line, U will be set to have all the items, and F will have itemHeaderNodes filled in:

U =	Index:	0	1	2	3	4	5	6	7
	Name:	—	a	b	c	d	e	f	g
	Left:	7	0	1	2	3	4	5	6
	Right:	1	2	3	4	5	6	7	0
F =	Index:	0	1	2	3	4	5	6	7
	Top:	—	0	0	0	0	0	0	0
	Up:	—	1	2	3	4	5	6	7
	Down:	—	1	2	3	4	5	6	7

Every subsequent line will only change the contents of F. For instance after processing the second line:

F =	Index:	0	1	2	3	4	5	6	7
	Top:	–	0	0	1	0	1	0	0
	Up:	–	1	2	9	4	10	6	7
	Down:	–	1	2	9	4	10	6	7
	Index:	8			9		10		11
	Top:	-1			3		5		-1
	Up:	–			3		5		9
	Down:	10			3		5		–

Note the nodes at indices 8 and 11 are spacer nodes. A similar process would be carried out for subsequent lines. Here is what F would look like after reading the next two lines of the input:

F =	Index:	0	1	2	3	4	5	6	7
	Top:	–	1	1	2	1	1	1	1
	Up:	–	12	16	17	13	10	18	14
	Down:	–	12	16	9	13	10	18	14
	Index:	8			9		10		11
	Top:	-1			3		5		-1
	Up:	–			3		5		9
	Down:	10			17		5		14
	Index:	11	12			13		14	15
	Top:	-1	1			4		7	-1
	Up:	9	1			4		7	12
	Down:	14	1			4		7	18
	Index:	15		16	17			18	19
	Top:	-1		2	3			6	-1
	Up:	12		2	9			6	16
	Down:	18		2	3			6	–

The spacer nodes are not replicated in F, but are shown twice for demonstration. Additionally, the order of the options or the order of the item names within an option do not change the performance of the exact cover solver but will lead to a different memory layout. A dash indicates that the field is not used.

2. Finding covers of U and F

To find and output covers of U and F, additional information needs to be stored such as which options were chosen at any point during the findCover method. This can be achieved by having an array of indices that keeps track of an OptionNode per option picked.

For this implementation, the amount of covers generated will be capped at maxSolutions. This is to avoid problems when inputting a partially filled in sudoku board that can have an impractically large number of valid solutions.

2.a. solve

public void solve(int maxSolutions) - sets up internal variables to track number of solutions then calls findCover(0)

private void findCover(int level) - finds the first maxSolution covers

1. If already found maxSolution covers then return
2. If U is empty
3. outputSolution(level)
4. i = item from U with minimum options associated with it
5. If i has zero options
6. return
7. cover(i)
8. For each option in F that has i
9. For each item j in option (where j != i)
10. cover(j)
11. Solution[level] = index of some node in current option
12. solve(level + 1)
13. For each item j in option (where j != i) // Note this is in reverse order to line 6.
14. uncover(j)
15. uncover(i)

2.b. cover/uncover

The cover and uncover methods allow for efficient manipulation of U and F.

- cover(int item) - hides all options in F that contain the item and removes the item from U
- uncover(int item) - restores item to U and unhides all options in F that contain the item
- hide(int p) - deletes all nodes in same option as p from respective vertical lists
- unhide(int p) - undeletes all nodes in same option as p
- int mrv() - returns the index of an uncovered item in U with the least number of options

The pseudocode for cover/hide, along with an example walkthrough of how the vertical pointers are changed after each method is called, can be found in the slides for reference. Note that unhide and uncover should traverse nodes in the reverse order as hide and cover.

2.c. outputSolution

For every solution found, the following output should be written to solution.txt formatted as such:

- **First line:** Every solution starts with a header line in the form:
 - "Solution #k"
 - k is the number of solutions found so far (starting from 1)
- **Subsequent lines:** Represent the options that make up the solution
 - The item names within any option can be displayed in any order
 - The options can be displayed in any order

Note: The solutions can be displayed in any order. To retrieve the item names for an option included in the solution, the solution array should be used. Every OptionNode referenced in the solution array refers to a distinct option. For each OptionNode, its associated option can be reconstructed by getting all OptionNodes contiguous to it in F and getting their ItemNames using the top field to get the index of their respective ItemHeaderNode. Included in the skeleton code is a folder showing examples of properly formatted output for a given instance of an exact cover problem.

3. Sudoku Reduction and output translation

Solving a Sudoku puzzle via Algorithm X requires reducing the puzzle into an exact cover instance, solving that instance, and then mapping each returned exact-cover solution back into a completed Sudoku grid.

3.a. Reducing Sudoku to an Exact Cover Instance

A standard 9x9 Sudoku puzzle can be encoded as an exact cover problem by defining items that enforce the four types of Sudoku constraints:

- Each cell (x, y) is assigned exactly one digit
- Each digit k appears exactly once in each row x
- Each digit k appears exactly once in each column y
- Each digit k appears exactly once in each 3x3 box z

The items, which is the same for every puzzle, includes the cell items (p00, p01, p02, ..., p88), the row items (r01, r02, r03, ..., r89), the column items (c01, c02, c03, ..., c89), and the box items (b01, b02, b03, ..., b89).

Each placement of the digit k into cell (x, y) corresponds to exactly one option: pxy rxk cyk bzk (note that $z = 3*(x/3) + y/3$).

To construct the exact cover instance given a sudoku puzzle, first list out the full item list: p00, p01, p02, ... p88, r01, r02, r03, ... r89, c01, c02, c03, ... c89, b01, b02, b03, ..., b89. Then for every cell already filled in, add the option pxy rxk cyk bzk where k is the filled in number. For every empty cell, add nine options, with k ranging from 1-9.

3.b. Exact cover output to Sudoku solution

Each solution returned by the exact cover solver should consist of 81 distinct options which correspond to a digit being placed in every cell of the grid. To reconstruct a Sudoku board from such a solution, locate the item of the form pxy in each option to determine the cell coordinates and recover the digit k from one of the remaining items (rxk, cyk, or bzk). Insert this digit into a 9x9 character array representing the board.

Once all placements are applied, the filled 9x9 grid is flattened into an 81-character string, using row-major order (row 0 to row 8, left to right within each row). Each completed board is stored as one entry in the String[] result array.

4. SudokuGUI

After implementing the exact cover solver and the sudoku solver, the provided SudokuGUI.java file can be used to test and visualize the system. The GUI displays a Sudoku board, allows puzzles to be loaded, and uses the methods defined in Section 3.